

Lazy Loading in EF Core

1. What does “loading” mean in EF Core?

Suppose you have a Student entity with a navigation property for StudentCourses. When EF Core retrieves a Student, it needs to determine whether the related courses should also be retrieved.

```
public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }

    public virtual ICollection<StudentCourses> StudentCourses { get; set; }
}
```

2. Eager Loading

Eager loading means explicitly telling EF Core to retrieve the related data at the same time as the main entity.

```
var student = context.Students
    .Include(s => s.StudentCourses)
    .First();
```

Conceptually: You ask for Student → EF gets Student → EF also gets StudentCourses → student.StudentCourses is populated.

3. Lazy Loading

With lazy loading, you do not explicitly request StudentCourses when retrieving the Student. The related data is loaded only when you actually access the navigation property.

```
var student = context.Students.First();

foreach (var sc in student.StudentCourses)
{
    // ...
}
```

With lazy loading enabled, EF Core notices that StudentCourses has not been loaded and automatically sends another query to retrieve it.

```
SELECT * FROM Students

-- later, when StudentCourses is accessed:
SELECT * FROM StudentCourses WHERE StudentId = 1
```

4. So why virtual?

For EF Core's lazy-loading proxies, virtual allows EF Core to create a special proxy/derived class that can intercept access to the navigation property.

Think of it as: Student object → EF proxy → you access StudentCourses → proxy loads the data from the database → the courses are returned.

Important: virtual by itself does NOT enable lazy loading. Lazy-loading proxies must also be configured.

5. Why can lazy loading be dangerous?

If you retrieve 100 students and then access `StudentCourses` for every student, lazy loading can result in one query for the students plus one query per student for the courses. This is known as the N+1 query problem.

```
Query 1:
SELECT * FROM Students

Query 2:
SELECT * FROM StudentCourses WHERE StudentId = 1

Query 3:
SELECT * FROM StudentCourses WHERE StudentId = 2

...

Query 101:
SELECT * FROM StudentCourses WHERE StudentId = 100
```

Using `Include` can avoid this pattern by explicitly loading the relationship as part of the query.

```
var students = context.Students
    .Include(s => s.StudentCourses)
    .ToList();
```

6. In YOUR project

```
public virtual ICollection<StudentCourses> StudentCourses
{
    get;
    set;
} = new HashSet<StudentCourses>();
```

ICollection<StudentCourses>: This means the `Student` has a collection of `StudentCourses`.

= new HashSet<StudentCourses>(): When a `Student` object is created, the collection is initialized and ready to use.

virtual: The property can be overridden, and EF Core can use it for proxy-based lazy loading if lazy loading is configured.

In your project, lazy loading is not enabled, so `virtual` does not cause automatic database queries.